

本地 LLM Agent KV Cache 复用实验报告

本地 Agent KV Cache Benchmark 项目
Qwen2.5-Coder-3B-Instruct / llama.cpp Metal
local-agent-kv-cache-benchmark

摘要

长上下文本地 Agent 在多轮交互中会反复提交 system prompt、工具定义、代码仓库摘要或文档片段，重复 prefill 会显著抬高首 token 延迟。本文基于本项目已完成的 benchmark，评估 llama.cpp 本地服务中的 prompt/KV cache 对 coding agent 与 RAG agent 的影响。实验在 Apple M5、32 GB unified memory、Qwen2.5-Coder-3B-Instruct Q4_K_M、单并发配置下运行，覆盖 2K、4K、8K、16K prompt 长度、三类缓存策略、动态字段位置敏感性和 `--cache-reuse` 参数 sweep。结果显示，只打开 cache 但把动态字段放在 prompt 前部几乎不能改善 warm TTFT；将稳定内容放在前缀后，coding workload 在 2K–16K 上获得 17.0× 到 112.2× TTFT 加速，RAG workload 获得 13.6× 到 103.7× 加速。8K layout 实验进一步表明，front-volatile 布局会使 warm TTFT 回到 9–12 秒量级，而 stable-prefix/end-volatile 布局能保持在 0.15–0.18 秒量级。相较之下，`--cache-reuse` 在当前单会话稳定前缀设置下不是主要杠杆，绝大多数组合相对 reuse=0 的变化位于约 $\pm 10\%$ 内。结论是：本地 Agent 的 prompt 组织方式比单纯开启缓存更关键；稳定、长、可复用的前缀应作为系统级接口约束。

1 引言

本地 LLM Agent 通常把系统指令、工具 schema、仓库摘要、相关文件片段、检索文档和历史摘要拼接成一个长 prompt。多轮请求之间，真正变化的内容往往只是当前用户问题、时间戳、request id、最新错误日志或少量对话状态。如果服务端不能复用前几轮已经 prefill 的前缀，每一轮都会重复计算大段相同上下文，首 token 延迟会随上下文长度快速上升。

本实验关注一个工程问题：在本地单机 Agent 服务中，prompt/KV cache 是否能把长上下文 warm turn 的 TTFT 降到接近短请求水平；如果能，prompt 应如何组织才能稳定触发复用。我们比较三类策略：S0 为关闭 prompt cache 的 baseline，S1 为开启 cache 但保留 front-volatile 布局，S2 为开启 cache 并把稳定内容集中在 prompt 前缀。随后用 8K layout sensitivity 实验隔离动态字段位置的影响，并用 cache-reuse sweep 检查 `--cache-reuse` 是否还能带来额外收益。

2 实验设置

2.1 硬件与软件环境

表 1: **实验环境**。所有正式实验均在同一台本机上顺序执行，固定单并发以降低 slot 调度和并发竞争带来的噪声。

项目	配置
机器	MacBook Pro, Mac17,2
芯片与内存	Apple M5, 10-core CPU, 10-core GPU, 32 GB unified memory
系统与架构	macOS 26.4.1 build 25E253, arm64
推理服务	llama-server version 9110, Metal enabled, OpenAI-compatible API
模型	qwen2.5-coder-3b-instruct-q4_k_m.gguf
Python 环境	Python 3.11.14, openai, psutil, pandas, matplotlib
请求设置	temperature=0, top_p=1, max_tokens=64, --parallel 1

2.2 Workload 与策略

实验使用可控合成 workload，目的是让不同策略之间的主要差异来自缓存可复用前缀，而不是任务内容差异。Coding workload 模拟本地代码助手：prompt 包含系统指令、工具定义、仓库结构、合成源文件、测试文件、对话摘要、动态元数据和当前用户请求。RAG workload 模拟长文档问答：prompt 包含固定文档上下文和逐轮变化的问题。

每个 sequence 含 20 轮请求。统计时将第 1 轮作为 cold turn，正文主要报告第 2–20 轮 warm turn 的中位数指标。主实验策略定义如下：

表 2: **被比较的缓存策略**。S1 与 S2 都打开 prompt cache，差异在于 prompt layout 是否保留可复用的稳定前缀。

编号	策略名	服务端参数	Prompt layout
S0	no_cache	--no-cache-prompt	stable_prefix 内容，但服务端关闭复用
S1	default_cache	--cache-prompt	front_volatile，动态字段位于最前部
S2	stable_prefix	--cache-prompt	系统指令、工具、代码或文档均稳定前置，动态字段放末尾
S3	cache-reuse sweep	--cache-prompt --cache-reuse N	基于 S2， $N \in \{0, 64, 128, 256, 512\}$

2.3 指标

核心指标是 TTFT，即请求发出到第一个非空 token 到达的时间。总延迟、decode latency、decode 吞吐、prompt token、completion token 和服务端 RSS 由 streaming client 与 psutil 采集。报告中的加速比统一定义为同 workload、同 prompt 长度下的

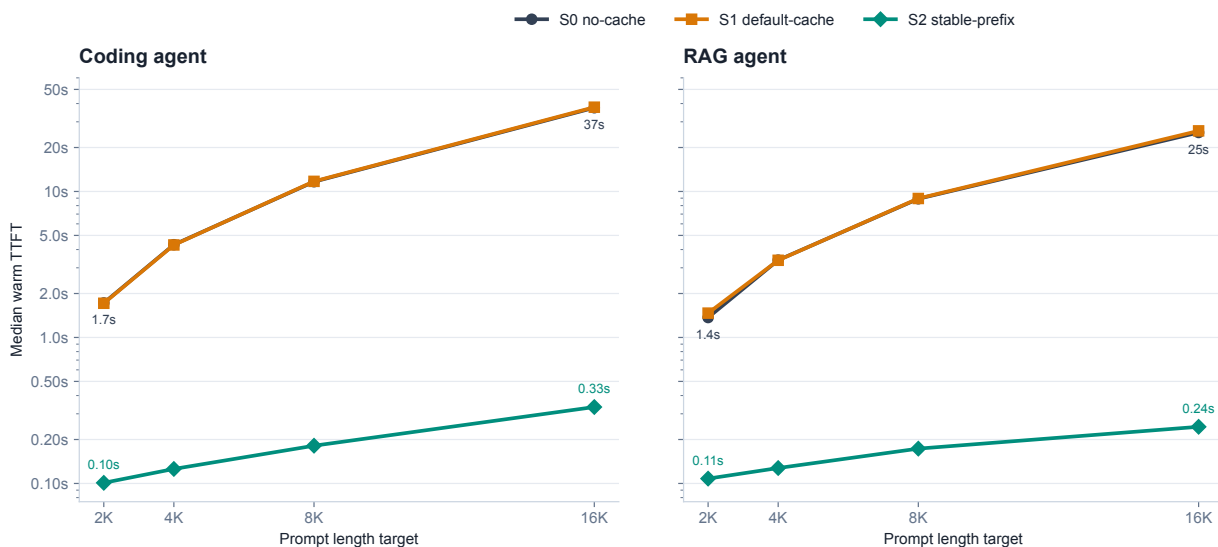
$$\text{speedup} = \frac{\text{median TTFT}_{S0}}{\text{median TTFT}_{\text{strategy}}}.$$

因此，大于 1 表示比关闭 cache 更快。

3 主实验结果

Warm TTFT collapses when the stable prefix is reusable

Median first-token latency across 20-turn sequences; y-axis is log-scaled to show 2K through 16K.

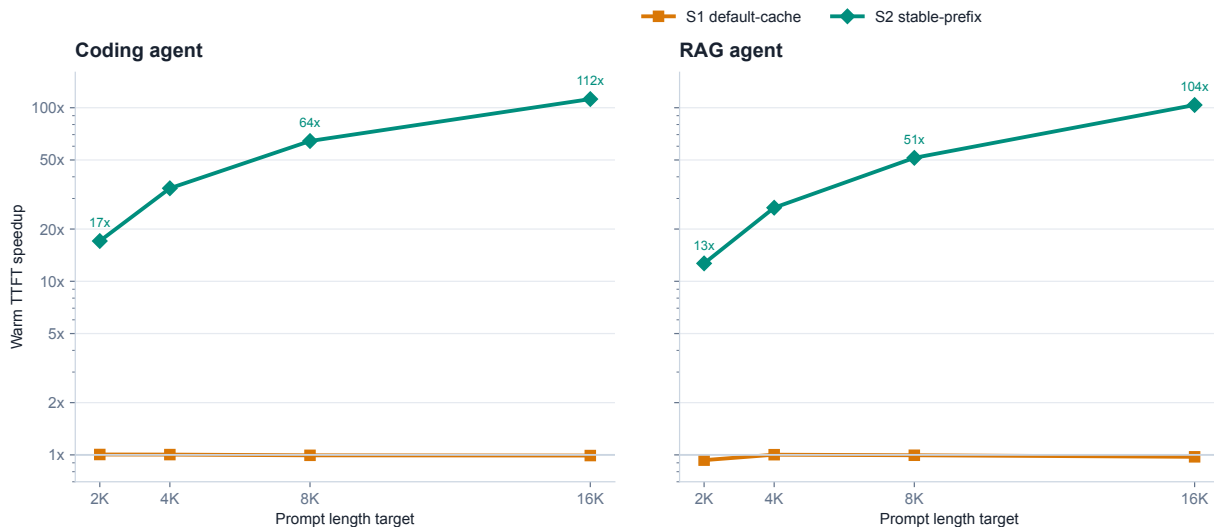


Qwen2.5-Coder-3B Q4_K_M, llama.cpp Metal, --parallel 1; warm turns only.

图 1: 不同 prompt 长度下的 warm TTFT。S2 stable-prefix 将 2K–16K 的 warm TTFT 压到 0.10–0.33 秒；S0 与 S1 随上下文长度增长显著变慢。

Stable-prefix prompt layout scales better than the context

TTFT speedup is computed against S0 no-cache for the same workload and prompt length.



S1 keeps cache enabled but uses front-volatile layout; S2 keeps the long prefix stable.

图 2: 相对 S0 的 TTFT 加速比。S1 由于动态字段位于 prompt 前部，几乎没有复用收益；S2 的收益随上下文长度增大而提高。

表 3: 主实验 warm-turn 指标。TTFT 与总延迟单位为秒，RSS 为服务端峰值 MiB；S2 加速比以同 workload、同长度的 S0 为 baseline。

Workload	长度	S0 TTFT	S1 TTFT	S2 TTFT	S2 总延迟	S2 RSS	S2 加速比
Coding	2K	1.724	1.715	0.101	1.249	2277.0	17.0×
Coding	4K	4.319	4.302	0.126	1.332	2432.2	34.4×
Coding	8K	11.661	11.728	0.181	1.549	2742.5	64.3×
Coding	16K	37.443	37.779	0.334	2.268	3361.6	112.2×
RAG	2K	1.369	1.468	0.108	0.457	2274.9	13.6×
RAG	4K	3.386	3.378	0.128	0.519	2427.8	26.5×
RAG	8K	8.908	8.952	0.173	0.574	2732.7	51.4×
RAG	16K	25.341	26.004	0.244	0.630	3342.7	103.7×

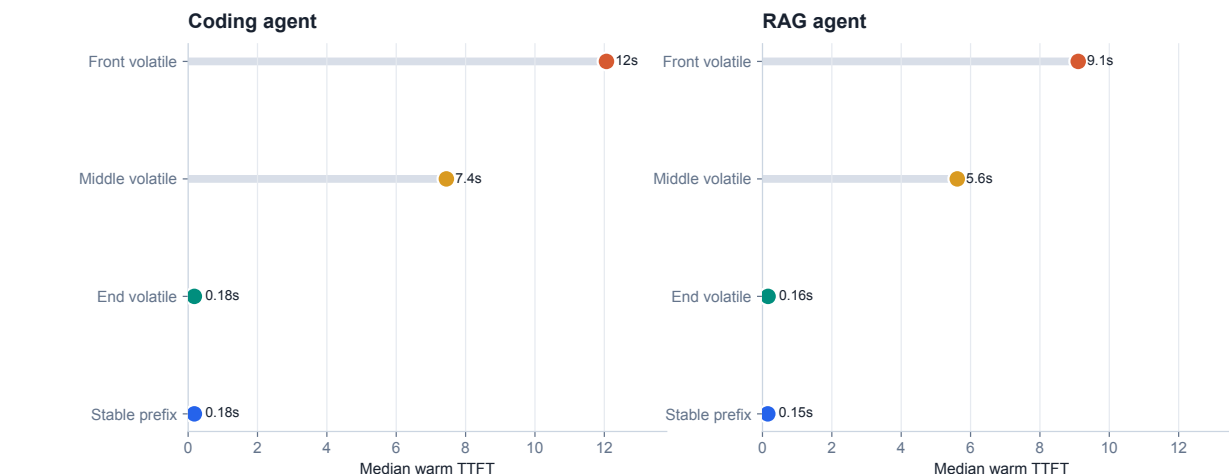
主实验结果有三个直接结论。第一，关闭 cache 时，TTFT 基本由长上下文 prefill 主导：coding 从 2K 的 1.72 秒升至 16K 的 37.44 秒，RAG 从 1.37 秒升至 25.34 秒。第二，单纯开启 cache 但让动态字段出现在最前部，不能产生稳定收益；S1 在 4K–16K 上与 S0 几乎重合，部分组合还略慢。第三，S2 的 warm TTFT 随长度增长非常缓慢，16K coding 仍只有 0.334 秒，16K RAG 为 0.244 秒。

总延迟的变化也符合预期。S2 减掉了重复 prefill，剩余延迟主要来自 decode。Coding workload 固定输出 64 token，因此 16K S2 总延迟仍约 2.27 秒；RAG 的回答更短，中位 completion token 约 17–18 个，因此 16K S2 总延迟约 0.63 秒。

4 Prompt layout 敏感性

Dynamic fields at the front erase most cache benefit

Same 8K task shape, same cache flag; only the location of volatile content changes.



End-volatile and stable-prefix both preserve a long reusable prefix; front-volatile does not.

图 3: 8K prompt 下动态字段位置对 warm TTFT 的影响。四组实验内容形态一致，只改变 volatile 内容的位置。

表 4: **8K layout sensitivity**。动态字段越靠前，可复用前缀越短，TTFT 越接近无缓存。

Workload	Layout	中位 TTFT (s)	相对 front-volatile 加速
Coding	front_volatile	12.062	1.0×
Coding	middle_volatile	7.446	1.6×
Coding	end_volatile	0.179	67.3×
Coding	stable_prefix	0.180	67.2×
RAG	front_volatile	9.104	1.0×
RAG	middle_volatile	5.616	1.6×
RAG	end_volatile	0.161	56.4×
RAG	stable_prefix	0.153	59.4×

Layout sensitivity 是本实验最关键的解释性结果。所有组都打开 `--cache-prompt`，但 front-volatile 会让每轮请求从第一个 token 开始就不同，服务端无法复用长前缀；middle-volatile 只能复用 volatile 字段之前的一段内容，因此收益有限。end-volatile 和 stable-prefix 都把动态内容推到末尾，保留了长且完全一致的前缀，TTFT 下降两个数量级。对实际 Agent 系统而言，这意味着时间戳、trace id、当前工具结果、最新错误日志和当前用户请求都不应插入 system prompt 或工具定义之前。

5 Cache-reuse 参数 sweep

Cache-reuse tuning is not the main lever here

Relative TTFT speedup versus reuse=0. Values near 1.0 mean no meaningful change.



Shaded band marks +/-5%. In this single-session stable-prefix setup, reuse tuning stays inside or near noise-level differences.

图 4: `--cache-reuse` 参数 sweep。图中展示相对 reuse=0 的 TTFT 加速比；灰色带为 $\pm 5\%$ 。

表 5: **Cache-reuse sweep 摘要**。当前单并发、稳定前缀、多轮顺序请求设置下，`--cache-reuse` 不是主要优化项。

Workload	长度	reuse=0 TTFT	最佳 TTFT	最差 TTFT
Coding	8K	0.181	0.181 (reuse=0)	0.203 (reuse=512)
Coding	16K	0.325	0.316 (reuse=64)	0.325 (reuse=0)
RAG	8K	0.152	0.152 (reuse=0)	0.171 (reuse=256)
RAG	16K	0.263	0.258 (reuse=256)	0.265 (reuse=128)

Sweep 结果显示，稳定前缀已经提供了主要收益后，额外调节 `--cache-reuse` 的边际影响较小。8K coding 和 8K RAG 中，非零 reuse 反而略慢；16K coding 的 reuse=64 比 reuse=0 快约 2.9%，16K RAG 的 reuse=256 快约 2.1%。这些差异明显小于 layout 带来的 50× 以上差距，因此在当前配置下，优先级应是规范 prompt 前缀，而不是优先搜索 reuse 参数。

6 内存与吞吐

RSS remains modest for the selected 3B Q4 local setup

Peak server RSS by workload, prompt length, and strategy. Values include llama-server process memory.



图 5: **不同策略的服务端峰值 RSS**。S1 front-volatile 组的 RSS 明显更高，反映了缓存开启但无法有效命中时更高的缓存保留成本。

内存结果表明，3B Q4 模型在 2K–16K 范围内适合当前 32 GB unified memory 机器。S2 的峰值 RSS 从约 2.28 GiB 增至约 3.36 GiB；S0 也处在相近水平。S1 的峰值 RSS 更高，8K coding 达 9622.7 MiB，16K RAG 达 10719.0 MiB，原因是 cache 已启用但 dynamic-front 布局使复用失败，服务端仍可能保留更多不可复用或低收益缓存状态。吞吐方面，decode tokens/s 主要受输出长度和模型 decode 速度影响，不是本实验的主矛盾；S2 的主要收益来自 prefill 被复用，而非 decode 阶段变快。

7 工程建议与局限

本实验支持以下工程建议。第一，把系统指令、工具定义、工具 schema、仓库摘要、固定文档上下文、长期记忆摘要等稳定内容放在 prompt 最前部，并保证跨轮字节级稳定。第二，把时间戳、request id、turn id、当前错误日志、工具返回、当前用户请求等动态内容集中放到 prompt 末尾。第三，Agent 框架应把 prompt

assembly 当作性能敏感路径处理，避免在稳定前缀中插入随机顺序的 JSON 字段、当前时间、临时路径、计数器或日志片段。第四，只有在稳定前缀已经满足后，再考虑 `--cache-reuse`、并发 slot、上下文长度和模型量化等更细粒度调参。

本报告也有局限。Workload 是合成的，能隔离变量，但不覆盖真实大型代码库、真实检索系统和多用户交替会话的全部复杂性。实验固定单并发，因此没有测量多 slot 并发下 cache 被抢占或污染的情况。模型固定为 Qwen2.5-Coder-3B Q4_K_M，结论中的绝对延迟不能直接外推到 7B、14B 或不同推理后端。不过，layout 实验只改变 volatile 字段位置，因果解释较强；“稳定前缀优先”的结论应当能迁移到多数 prompt-cache 实现。

8 结论

在本机 llama.cpp + Qwen2.5-Coder-3B 的实验条件下，prompt/KV cache 能把长上下文本地 Agent 的 warm TTFT 从秒级到几十秒级降低到百毫秒级，但前提是 prompt 拥有长且稳定的可复用前缀。仅打开 cache 不足以获得收益；如果动态字段位于 prompt 前部，性能几乎等同于关闭 cache，并可能增加 RSS。对本地 coding agent 和 RAG agent，最有效的优化不是先调 `--cache-reuse`，而是建立稳定前缀协议：固定系统指令、工具定义和长上下文的顺序与内容，把所有 volatile 信息后置。